

Operating System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- **Internal structure of different Operating Systems can vary widely**
- Start the design **by defining goals and specifications**
- Affected by **choice of hardware, type of system**

Operating System Design and Implementation

- **User** goals and **System** goals
 - User goals – operating system should be
 - convenient to use,
 - easy to learn,
 - reliable,
 - safe, and
 - fast

Operating System Design and Implementation

- System goals – operating system should be
 - easy to design, implement, and maintain,
 - as well as flexible,
 - reliable,
 - error-free, and
 - efficient

Operating System Design and Implementation (Cont.)

- Important principle to separate

Policy: *What* will be done?

Mechanism: *How* to do it?

- Policies decide what will be done, Mechanisms determine how to do something
- **The separation of policy from mechanism is a very important principle,**
 - it allows maximum **flexibility if policy decisions are to be changed later (example – timer)**
- Specifying and designing an OS is highly creative task of **software engineering**

Implementation

- Much variation
 - Early OSes in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
 - But slower

OS Design Considerations for Multiprocessor and Multicore architectures

Symmetric Multiprocessor

- In an SMP system,
 - the **kernel can execute on any processor**, and
 - typically each processor does self-scheduling from the pool of available processes or threads.
 - The kernel can be constructed
 - **as multiple processes or**
 - **as multiple threads,**
 - allowing **portions of the kernel to execute in parallel.**

Symmetric Multiprocessor OS

- In an SMP system,
 - The SMP approach complicates the OS.
 - The OS designer must deal with the complexity due to
 - sharing resources (like data structures) and
 - coordinating actions (like accessing devices)
 - from multiple parts of the OS executing at the same time.

OS Design Considerations for Symmetric Multiprocessor OS:

- 1) Simultaneous concurrent processes or threads**
- 2) Scheduling**
- 3) Synchronization**
- 4) Memory management**
- 5) Reliability and fault tolerance**

OS Design Considerations for Symmetric Multiprocessor OS:

Simultaneous concurrent processes or threads:

- **Kernel routines need to be reentrant**
 - to allow several processors
 - to execute the same kernel code simultaneously.
- With multiple processors executing the same or different parts of the kernel,
 - kernel tables and management structures must be managed properly
 - to avoid data corruption or invalid operations.

OS Design Considerations for Symmetric Multiprocessor OS:

Scheduling:

- Any processor may perform scheduling,
 - which complicates the task of enforcing a scheduling policy and
 - **assuring that corruption of the scheduler data structures is avoided.**
- If kernel-level multithreading is used, then
 - The opportunity exists to schedule **multiple threads from the same process simultaneously on multiple processors.**

OS Design Considerations for Symmetric Multiprocessor OS:

Synchronization:

- With multiple active processes having potential access
 - to shared address spaces or shared I/O resources,
 - care must be taken to provide effective synchronization.
- **Synchronization is a facility that enforces**
 - mutual exclusion and
 - event ordering.
- **A common synchronization mechanism**
 - used in multiprocessor operating systems is locks.

OS Design Considerations for Symmetric Multiprocessor OS:

Memory management:

- Memory management on a multiprocessor must deal with
 - all of the issues found on uniprocessor computers
- In addition, the OS needs to exploit
 - the available hardware parallelism to achieve the best performance.

OS Design Considerations for Symmetric Multiprocessor OS:

Memory management:

- The paging mechanisms on different processors must be coordinated to enforce consistency
 - when several processors share a page or segment and
 - to decide on page replacement.
 - The reuse of physical pages is the biggest problem of concern;
 - **that is, it must be guaranteed**
 - **that a physical page can no longer be accessed with its old contents**
 - **before the page is put to a new use.**

OS Design Considerations for Symmetric Multiprocessor OS:

Reliability and fault tolerance:

- The OS should provide graceful degradation
 - in the face of processor failure.
- The scheduler and other portions of the OS
 - must recognize the loss of a processor and
 - restructure management tables accordingly

OS Design Considerations for Symmetric Multiprocessor OS:

Reliability and fault tolerance:

- Because multiprocessor OS design issues generally involve extensions to
 - solutions to multiprogramming uniprocessor design problems,
 - we do not treat multiprocessor operating systems separately.

OS Design Considerations for Multicore architectures

- The considerations for multicore systems include
 - all the design issues discussed so far in this section for SMP systems.
- But additional concerns arise.
 - The issue is one of the scale of the potential parallelism.

Multicore architectures

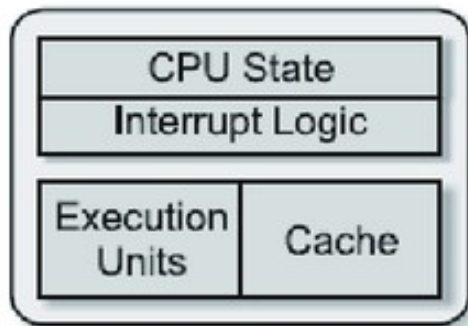
- Current multicore vendors offer **systems with up to eight cores on a single chip.**
- With each succeeding processor technology generation,
 - **the number of cores and**
 - **the amount of shared and dedicated cache memory increases,**
 - **so that we are now entering the era of “many-core” systems.**

Unicore vs Multicore

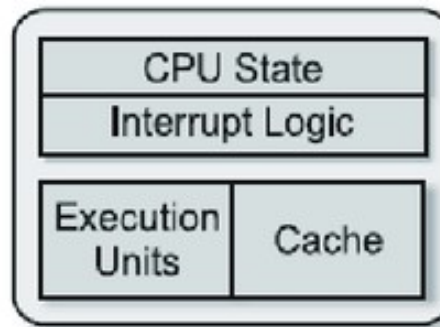
- **A processor with a single core is called a Unicore processor.**

- **But a processor with two or more cores is called a Multicore processor.**

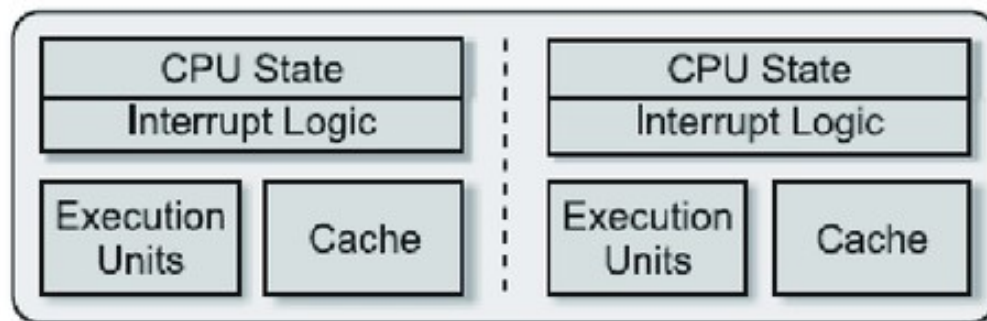
Multicore



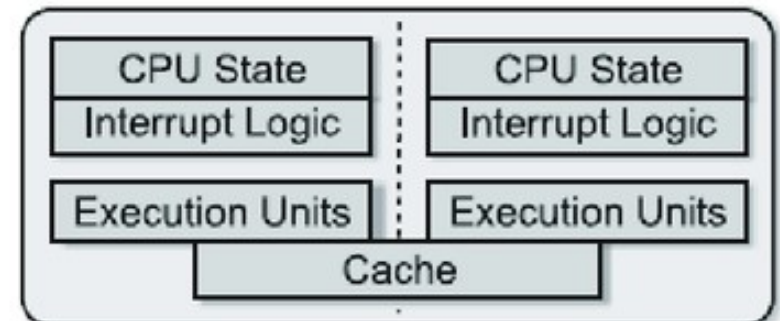
(a) Single core



(b) Multiprocessor



(c) Multi-core

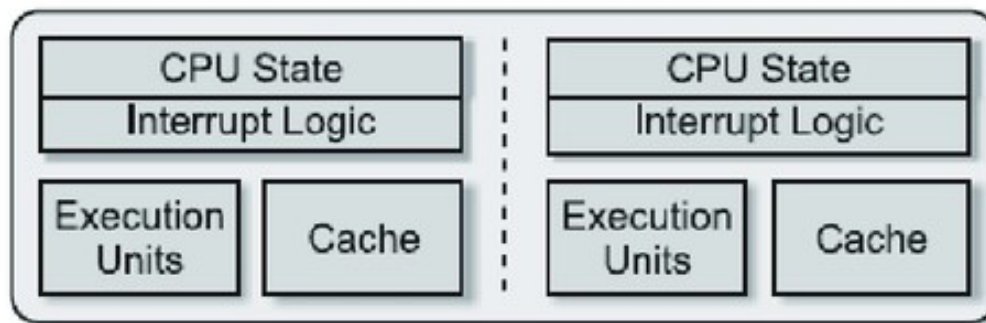


(d) Multi-core with shared cache

https://www.researchgate.net/figure/Simple-comparison-of-single-core-multi-processor-and-multi-core-architectures-after_fig3_266489075

Multicore

- The cores of a multicore processor can individually read and execute program instructions at the same time.
- This increases the speed of execution of the programs and supports parallel computing.
- Eg: Quadcore processors, Octacore processors, etc.



(c) Multi-core

OS Design Considerations for Multicore architectures

PARALLELISM WITHIN APPLICATIONS

- Most applications can be subdivided into multiple tasks that can execute in parallel,
 - With these tasks subdivided into multiple processes,
 - Perhaps each with multiple threads.
- The difficulty is that the developer must decide
 - how to **split up the application work into independently executable tasks.**
 - That is, the developer must decide **what pieces can or should be executed asynchronously or in parallel.**

OS Design Considerations for Multicore architectures

PARALLELISM WITHIN APPLICATIONS

- It is **primarily the compiler and the programming language features**
 - that **support the parallel programming** design process.
- But, the OS can support this design process, at minimum,
 - by efficiently allocating resources among parallel tasks as defined by the developer.

OS Design Considerations for Multicore architectures

VIRTUAL MACHINE APPROACH

- With the **ever-increasing number of cores on a chip**,
 - the attempt to multiprogram individual cores to support multiple applications may be a misplaced use of resources.

OS Design Considerations for Multicore architectures

VIRTUAL MACHINE APPROACH

- If instead, **we allow one or more cores to be dedicated to a particular process** and
 - then leave the processor alone to devote its efforts to that process,
 - we avoid much of the overhead of task switching and scheduling decisions.

OS Design Considerations for Multicore architectures

VIRTUAL MACHINE APPROACH

- The **multicore OS could then act as a hypervisor**
 - that makes a high-level decision **to allocate cores to applications**
 - but does little in the way of resource allocation beyond that.